Enemy Behaviour System

Contents

- How an Enemy Is Put Together
- Making a New Enemy
- Enemy Settings
- How Enemies Choose What to Do
- Animator
- Movement Scripts
- Attacks
- Stagger
- Bosses
- Extending the System
- Gotchas

Every enemy in the game, from basic enemies up to the boss fight, is built from a few components combined on a prefab. Most new enemy types can be made in the Inspector without writing code.

See also: Enemy Encounter & Spawning, Special Shot, Shaders & VFX (stagger bar, Danger Volumes), Editor Scripts (`ShowIf`).

How an Enemy Is Put Together

Part	Component	Job
Brain	`Enemy`	Decides what to do: chase, attack, wait, die.
Legs	`GroundMovement` or `FlyingMovement`	Moves the enemy when the brain asks. Leave out for a stationary enemy.
Attacks	`EnemyAttack_Melee`, `_AoE`, `_Ranged`, `_RangedMulti`, `_Charge`, `_Kamikaze`	Each runs its own windup, strike and recovery. Add as many as you like.
Body	`EnemyStagger`	Takes the player's shots, builds stagger, shows the stagger bar.
Death condition	`WeakPointManager`	Tracks weakpoint cycles, which decide when the enemy dies.

The brain never moves the enemy or deals damage itself, so movement and attacks can be swapped freely.

> **In code:** `Enemy` talks to movement only through the `IEnemyMovement` interface (`Chase`, `Strafe`, `Stop`, `FaceTarget` and so on), and to attacks only through `EnemyAttack_Base` (`PerformAttack`, `CancelAttack`, `IsReady`, `IsAttacking`). Shots reach the enemy through `EnemyStagger`, which implements `IDamageable`.

Making a New Enemy

1. Add **Enemy** to the enemy's root. It automatically adds an **EnemyStagger** and picks up any Animator, movement script and attacks already on the object.
2. Add a movement script: **GroundMovement** (adds a NavMeshAgent) or **FlyingMovement** (adds a Rigidbody). Skip this for a stationary enemy.
3. Add attack components and order them in the **Attacks** list. **Top = highest priority** (see How Enemies Choose What to Do).
4. Set up **EnemyStagger** and assign its **Weak Point Manager**.
5. Optional extras:
    - **Animator** with the triggers listed below.
    - **SoundPlayer**. Sound index **1** plays when the enemy is hit while immune.
    - **Blood / Immune Fx Emitters** for hit particles.
    - **DissolveEffect** in its children to dissolve on death.
    - **BestiaryTracker** to appear in the Bestiary.

> The auto-fill in step 1 only happens when Enemy is first added (or reset). Attacks added later need adding to the list by hand.

The player must be tagged **Player**.

Enemy Settings

Field	What it does
Enemy Class	Decides how the enemy dies. See Enemy Classes.
Skip Spawn / **Spawn Delay**	How long the enemy spends spawning. It can't be hit or act during this.
Immune To Special Shot	Blocks the Special Shot unless its Special weakpoint is hit. See Special Shot.
Always Aggro / **Aggro Range**	When the enemy notices the player (yellow gizmo).
Movement Script	The movement component. Empty = stationary.
Attacks	Priority list of attacks.

Field	What it does
Champion settings	*(Champion only)* Number Of Phases, plus optional summons on chosen cycles.
Debug Mode	Logs the state machine and shows the live state in the Inspector.

Enemy classes

Class	Dies when
Standard	One full weakpoint cycle is completed.
Champion	**Number Of Phases** cycles are completed. Can summon enemies on chosen cycles. The Special Shot staggers it instead of killing it.
Thrall	It takes any hit at all.
Boss	Set automatically by the boss scripts. See Bosses.

> **In code:** the class rules live in `Enemy.CheckDie()`, which reads `stagger.weakPointManager.CyclesComplete` each frame. In `Awake`, `Enemy` sets `handleOwnDestruction = false` on the WeakPointManager so the enemy always handles its own death, and `dieOnWeakpointsComplete = false` for Champions and Bosses so their weakpoints reset for the next cycle.

Calling into an enemy from other scripts

Member	Use
`Die()`	Kills the enemy: plays blood, disables colliders, reports to its spawner, then dissolves (or is destroyed). Safe to call twice.
`SetPaused(bool)`	Freezes the brain and movement. Used by the Encounter Manager.
`CurrentState`, `IsDying`, `IsPaused`	Read-only state.
`OnDied`	Event raised when `Die()` starts. Used by `BestiaryTracker`.
`ImmuneHit`	Event raised when the enemy is shot while immune. Used by `TutorialManager`.

How Enemies Choose What to Do

- **The enemy fires the top-most attack in its list that's off cooldown and in range.**
- **If it can't fire anything, it moves to the range of the top-most attack that's off cooldown.** So the attack at the top of the list decides where the enemy likes to stand.
- **If every attack is on cooldown, it holds its distance and keeps facing the player.**

- **When staggered, it cancels its attack.** The cancelled attack goes on double cooldown.

"In range" means between the attack's **Min Range** and **Attack Range**.

> **In code:** `RefreshAttackCache()` runs at the top of `Update` and stores two picks: `selectedAttack` (ready and in range, what it fires) and `readyAttack` (ready at any range, what it moves towards). `GetEngagedState()` is the single place that decides between Chasing, Waiting and Idling. Max range has a 0.5 unit tolerance (`RangeTolerance`).

Preferences: some attacks only *prefer* certain situations (the AoE's *Prefer Stationary Targets*, the ranged attack's *Require Line Of Sight*). If no attack in range meets its preference, the enemy still uses the top-most one that's ready and in range.

> **In code:** `SelectAttack()` makes two passes. The first requires `ShouldUse(target)` to return true. The second ignores it. To make a rule strict instead of a preference, override `CanUseAttack()` in an `Enemy` subclass (this is how the bosses gate attacks by phase).

Movement during attacks: the enemy stops when an attack starts, unless the attack has **Move While Winding Up** ticked (it then keeps strafing at that attack's range). It always stops for the strike itself.

Animator

Trigger	Plays when
`idle`	Idle or waiting
`chase`	Chasing, returning or retreating
`windUp`	An attack is winding up
`attack`	An attack is striking
`stun`	Staggered
`spawn`	Spawning
`downed`	Downed (Sub-boss only)

Spawn and windup animations are sped up or slowed down to match **Spawn Delay** and the attack's **Windup Duration**, so they can be authored at any length.

> **In code:** `Enemy.Animations()` only calls `SetTrigger` when the wanted trigger changes (tracked in `lastAnimTrigger`). Calling `SetTrigger` every frame queues duplicate triggers that replay animations, so keep this guard if you add new states.

Movement Scripts

Both scripts share the same sections:

Section	Key fields
Chase	Chase Enabled, speed, **Only Chase If Attack Ready**
Chase Territory	**Chase Range** from the spawn point before giving up, or **Never Give Up Chase**
Return	Walk back to the spawn point after giving up
Retreat	Back away after every attack
Strafe	Circle the player, flipping direction every **Strafe Direction Interval**

Strafe Radius is only used by enemies with no attacks. Otherwise the attacks' ranges decide distance.

	GroundMovement	FlyingMovement
Uses	NavMeshAgent	Rigidbody (gravity is turned off automatically)
Chase speed	Ramps from **Min** to **Max Chase Speed** over **Chase Speed Ramp Distance**	**Chase Speed** with **Acceleration**
Extra	**Chase Easing** (0 sluggish, 1 snappy)	**Hover Height**, **Vertical Offset**, **Altitude Correction Speed**, **Ground Layers**, **Bobbing**

> **In code:** `GroundMovement` sets `navAgent.updateRotation = false` and rotates the enemy itself in `FaceTarget`. `FlyingMovement` works out horizontal and vertical velocity separately in `FixedUpdate` and then combines them, so altitude correction never slows down strafing.

Attacks

Shared settings

Every attack has **Min Range**, **Attack Range**, **Cooldown Time**, an optional **Windup Indicator** sprite (shown automatically during windup) and **Move While Winding Up**.

Ranges show as gizmos when the enemy is selected: red for max range, orange for min range.

Cancelling: when an enemy is staggered mid-attack, `CancelAttack()` stops the attack, cleans up any damage field or telegraph it spawned, and calls `StartCooldown(2f)` for a double-length cooldown.

Attack types

Attack	Use it for	Key settings
Melee	A telegraphed swing in front of the enemy. The player dodges by sidestepping during the windup.	Damage Field Prefab, Windup Duration, **Windup Turn Speed** (lower = easier to dodge), Strike/Recovery Duration, Damage
AoE	A ground-targeted area attack. A telegraph appears, then the damage lands in the same spot.	Damage Field Prefab, **Danger Zone Prefab**, Spell Radius, **Telegraph Duration** (the dodge window), Damage
Ranged	A single projectile.	Projectile Prefab, its stats (damage, speed, lifetime), Launch Point, **Require Line Of Sight**
Ranged Multi	A burst of several different projectiles.	Projectile Prefabs, **Selection Mode** (Sequence or Random), **Time Between Shots**
Charge	A straight-line charge through where the player was. Works for ground and flying enemies.	Charge Speed, **Charge Past Distance**, **Use Body Collider** or a Damage Field, **Use Back Off**
Kamikaze	Rushes the player and dies on contact.	Rush Speed, Contact Radius, Damage

AoE targeting: if the player is standing still, the AoE lands slightly in front of them (towards the enemy). If they're moving, it leads them with some random scatter. This keeps it readable without being dodged by just walking.

Ranged vs Ranged Multi: Ranged sets the projectile's stats from the attack. Ranged Multi uses the stats on each projectile prefab, so each projectile in the burst can behave differently.

Scripted AoEs: `EnemyAttack_AoE.PerformAttack(Vector3 position)` fires the AoE at a fixed point instead of at the player, for traps or scripted moments.

Damage Fields, Danger Zones and Projectiles

Prefab	Needs	Notes
DamageField	A **trigger** collider sized to 1 unit, plus the `DamageField` component	Scaled to the attack's radius and height. Settings come from the attack (damage, **Persist After Hit**, **Damage Over Time**).
DangerZone	The telegraph visual, plus `DangerZone`	Scaled to the AoE size unless **Scale To Radius** is off. Usually uses a Danger Volume shader.
Projectile	A **Rigidbody**, a **trigger** collider, plus `Projectile`	**Hit Layers** should leave out Enemy. **Destroyable By** sets which shot types can shoot it down.

Projectile damage fields: tick **Spawn Damage Field** on a projectile to leave a damage field on the ground where it hits. It spawns under the player on a direct hit, or on the floor below the impact point when it hits a wall. Set the projectile's own Damage to 0 if only the field should hurt.

> **In code:** damage is dealt by finding an `IDamageable` on the hit collider or its parents and calling `TakeDamage(DamageInfo)`. `DamageInfo` carries the amount, hit point, direction, source object and (for player shots) the shot type.

Stagger

- Each hit on the enemy's body adds to its stagger. At **Hits To Stagger**, it's staggered and its weakpoints appear.
- With **Stun On Windup** on, **one** hit during an attack windup staggers it straight away.
- Stagger drains over time at **Stagger Resistance** (higher = harder to stagger).
- A stagger lasts **Stagger Time**, plus **Time Added On Hit** each time a weakpoint is destroyed.
- **Immune To Bullets** makes the enemy shrug off shots entirely.

The stagger bar fills from the centre out using the MiddleOutFill shader. If its references are left empty, it finds a child Image named **Fill** and uses its parent as the bar.

Stagger from code

Member	Use
`TriggerStagger()`	Staggers immediately. Used by the Special Shot and the dash.
`ForceEndStagger()`	Ends the stagger now, running the normal exit. Used by the bosses.
`SetImmune(bool)`	Runtime immunity. Used by the Downed Sub-boss.
`OnStaggerStart`, `OnStaggerEnd`	Events. `HobgoblinTeleport` and `Enemy_BossMain` listen to `OnStaggerEnd`.
`canBeHit`	Off while spawning and dying.

Bosses

The boss fight is a **Main boss** and a **Sub-boss** fought together. Both are normal enemies with an extra script, so movement, attacks and stagger all work the same way.

Setup

1. Use **Enemy_BossMain** on the main boss and **Enemy_BossSub** on the sub-boss, in place of `Enemy`.
2. Keep them as separate prefabs. Leave the Main's **Sub Bosses** list empty and they'll link to each other automatically when spawned, in either order.
3. Spawn them from the boss room (see Enemy Encounter & Spawning).
4. Add a `downed` trigger to the Sub-boss's Animator.

How the fight works

- **Each phase is one full weakpoint cycle** on the Main boss. Finishing the last phase kills it.
- **When the Main changes phase**, its stagger ends, it summons that phase's minions, and the Sub-boss is released.
- **When the Main recovers from a stagger**, it summons minions and releases the Sub-boss.
- **The Sub-boss never dies from weakpoints.** Finishing a cycle knocks it **Downed**: it stops attacking and becomes immune until the Main releases it (or its **Downed Duration** runs out).
- **When the Main dies**, the Sub-boss and all minions die with it.
- **The Special Shot** staggers either boss in one hit.

> **In code:** both boss scripts inherit from the abstract `Enemy_Boss`, which overrides `CheckDie()` to call `OnCycleComplete(cycles)` once per finished cycle, and overrides `CanUseAttack()` to filter by the current phase's Allowed Attacks. The Main raises `PhaseChanged(int)` on each new phase, free for music or UI to listen to. Auto-linking uses static lists of active Mains and Subs (`Enemy_BossMain.FindAutoLinkMain()`, `Enemy_BossSub.GetActiveSubs()`).

Main boss settings

Field	What it does
Phases	One entry per phase. Each has **Allowed Attacks** (empty = all), **Summons On Phase Start** and **Summons On Stagger Recovery**.
Minion Prefab	What gets summoned.
Minion Spawn Radius / Height	Where minions appear around the boss (magenta gizmo ring).
Minion Blocking Layers / Clearance	Keeps minions from spawning inside walls.
Max Active Minions	Cap on living minions. 0 = no cap.

Sub-boss settings

Field	What it does
Phases	Allowed Attacks for each of the **Main boss's** phases.
Downed Duration	Gets back up on its own after this long. 0 = only the Main can release it.

Extending the System

New attack

Inherit from `EnemyAttack_Base` and follow this shape:

```
// Summary: Example attack. Winds up, does something, recovers.
public class EnemyAttack_Example : EnemyAttack_Base
{
    [SerializeField] private float windupDuration = 0.5f;
    [SerializeField] private float recoveryDuration = 0.3f;

    private bool isAttacking, isWindingUp;
    private Coroutine routine;

    public override bool IsAttacking => isAttacking;
    public override bool IsWindingUp => isWindingUp;
    public override float WindupDuration => windupDuration;

    public override void PerformAttack(Transform target)
    {
        if (!isAttacking) routine = StartCoroutine(Sequence(target));
    }

    public override void CancelAttack()
    {
        if (!isAttacking) return;
        StopCoroutine(routine);
        // clean up anything the attack spawned
        isAttacking = isWindingUp = false;
        InvokeAttackCancelled();
        StartCooldown(2f);
    }

    private IEnumerator Sequence(Transform target)
    {
        isAttacking = isWindingUp = true;
        InvokeWindupStart();
        yield return new WaitForSeconds(windupDuration);

        isWindingUp = false;
        InvokeStrikeStart();
        // the attack itself goes here
        InvokeStrikeEnd();

        yield return new WaitForSeconds(recoveryDuration);
        isAttacking = false;
        StartCooldown();
    }
}
```

- Keep `IsAttacking` true until the attack is completely finished.
- Always call `StartCooldown()` at the end and clean up in `CancelAttack()`.
- Override `ShouldUse(target)` to add a preference.
- Attack events (`OnWindupStart`, `OnStrikeStart`, `OnStrikeEnd`, `OnAttackCancelled`) are there for animation, sound and VFX scripts to listen to.

New movement type

Implement `IEnemyMovement` and drop the component into **Movement Script**. Use `GroundMovement` or `FlyingMovement` as a template.

Custom enemy logic

Inherit from `Enemy` (as the boss scripts do) and override `CheckDie()`, `CanUseAttack()`, `HandleSpecialShotHit()` or `Die()`. Always call the base version of `Awake()` and `Die()`.

Don't declare `Update`, `Reset`, `OnDestroy` or `OnDisable` in a subclass. They would replace `Enemy`'s own and break it.

Gotchas

- **The Attacks list is a priority list.** It controls both what fires and where the enemy stands.
- **Damage fields and projectiles need trigger colliders**, and the right target or hit layers.
- **Animator triggers must match the names above** exactly.
- **If a Champion dies after one cycle**, check its Enemy Class.
- **Strafe Radius does nothing on enemies with attacks.**